



Journal of Graphics Tools

Publication details, including instructions for authors and subscription information:

<http://www.tandfonline.com/loi/ujgt19>

Efficient Implementation of Marching Cubes' Cases with Topological Guarantees

Thomas Lewiner^a, H ello Lopes^b, Ant onio Wilson Vieira^c & Geovan Tavares^d

^a Pontifical Catholic University of Rio de Janeiro, Departamento de Matematica, Rua Marqu es de S ao Vicente, 225, Gavea 22453-900, Rio de Janeiro, Brazil E-mail:

^b Pontifical Catholic University of Rio de Janeiro, Departamento de Matematica, Rua Marqu es de S ao Vicente, 225, Gavea 22453-900, Rio de Janeiro, Brazil E-mail:

^c Pontifical Catholic University of Rio de Janeiro, Departamento de Matematica, Rua Marqu es de S ao Vicente, 225, Gavea 22453-900, Rio de Janeiro, Brazil E-mail:

^d Pontifical Catholic University of Rio de Janeiro, Departamento de Matematica, Rua Marqu es de S ao Vicente, 225, Gavea 22453-900, Rio de Janeiro, Brazil E-mail:

Published online: 06 Apr 2012.

To cite this article: Thomas Lewiner , H ello Lopes , Ant nio Wilson Vieira & Geovan Tavares (2003) Efficient Implementation of Marching Cubes' Cases with Topological Guarantees, Journal of Graphics Tools, 8:2, 1-15, DOI: [10.1080/10867651.2003.10487582](https://doi.org/10.1080/10867651.2003.10487582)

To link to this article: <http://dx.doi.org/10.1080/10867651.2003.10487582>

PLEASE SCROLL DOWN FOR ARTICLE

Taylor & Francis makes every effort to ensure the accuracy of all the information (the "Content") contained in the publications on our platform. However, Taylor & Francis, our agents, and our licensors make no representations or warranties whatsoever as to the accuracy, completeness, or suitability for any purpose of the Content. Any opinions and views expressed in this publication are the opinions and views of the authors, and are not the views of or endorsed by Taylor & Francis. The accuracy of the Content should not be relied upon and should be independently verified with primary sources of information. Taylor and Francis shall not be liable for any losses, actions, claims, proceedings, demands, costs, expenses, damages, and other liabilities whatsoever or howsoever caused arising directly or indirectly in connection with, in relation to or arising out of the use of the Content.

This article may be used for research, teaching, and private study purposes. Any substantial or systematic reproduction, redistribution, reselling, loan, sub-licensing, systematic supply, or distribution in any form to anyone is expressly forbidden. Terms & Conditions of access and use can be found at <http://www.tandfonline.com/page/terms-and-conditions>

Efficient Implementation of Marching Cubes' Cases with Topological Guarantees

Thomas Lewiner, Hélio Lopes, Antônio Wilson Vieira, and Geovan Tavares
Laboratório MatMídia, PUC – Rio de Janeiro, Brazil

Abstract. Marching Cubes methods first offered visual access to experimental and theoretical volumetric data. The implementation of this method usually relies on a small look-up table; many enhancements and optimizations of Marching Cubes still use it. However, this look-up table can lead to cracks and inconsistent topology. This paper introduces a full implementation of Chernyaev's technique to ensure a topologically correct result, i.e., a manifold mesh, for any input data. It completes the original paper for the ambiguity resolution and for the feasibility of the implementation. Moreover, the cube interpolation provided here can be used in a wider range of methods. The source code is available online.

1. Introduction

Isosurface extractors and implicit surface tilers opened up visual access to experimental and theoretical volumetric data, such as medical images, mechanical pieces, sculpture scans, mathematical surfaces, and physical simulation by finite elements methods. Among those techniques, the Marching Cubes [Lorensen, Cline 87] produces a surface out of a sampling of a scalar field $f : \mathbb{R}^3 \rightarrow \mathbb{R}$. It has been enhanced to a wide range of applications, from geological reconstruction [Tavares et al. 03], and medical images to three-dimensional scanning (see [Davis et al. 02] for an original use in the Digital

Michelangelo Project). Although this paper focuses on surface reconstruction from sampled data, the tilings of cubes introduced here can be used in simple reconstruction methods for synthetic data [Bloomenthal 94, Velho 96] in order to guarantee the topological consistency of the result when the precision of the result is limited.

1.1. *Marching Cubes*

Marching Cubes [Lorensen, Cline 87] has become the reference method when the sampled scalar field is structured on a cuberille grid. It classifies vertices as positive or negative, according to their comparison with a given isovalue. Then, it uses a look-up table to tile the surface inside the cube. This method has been enhanced and generalized in various directions, especially to reduce the number of cubes to be evaluated. However, most of those modern techniques still use a simple look-up table, which does not ensure the topological consistency of the result.

1.2. *Prior Work*

The main obstacles of the Marching Cubes' derived methods are the ambiguities inherent to data sampling. Those ambiguities can appear on the faces of a cube, or inside the cube. The ambiguities on faces have been resolved in [Nielson, Hamann 91], assuming the scalar field f is trilinear over each cube, which gave a modified look-up table [Montani et al. 94]. Using the same hypothesis, it is possible to resolve internal ambiguity as done in [Natarajan 94, Chernyaev 95]. Further approaches [Gelder, Wilhelms 94, Bajaj et al. 98, Weber et al. 02] compute the topology of f as a volume, giving rise to more complex algorithms that need, at the end, to tile each cube.

1.3. *Contribution*

In this paper, we describe an efficient and robust implementation of Chernyaev's Marching Cubes 33 algorithm [Chernyaev 95] (see Figure 1). We needed to complete Chernyaev's paper on the internal ambiguity resolution. We computed and tested the 730 subcases of the enhanced look-up table. This table can be used *as is* in Marching Cubes' improvements (in particular, those that avoid empty cell tests). Our result is guaranteed to be a manifold surface, with no crack, with the topology of the trilinear interpolation of the scalar field over each cube. The complete source code is available online at the address listed at the end of this paper.

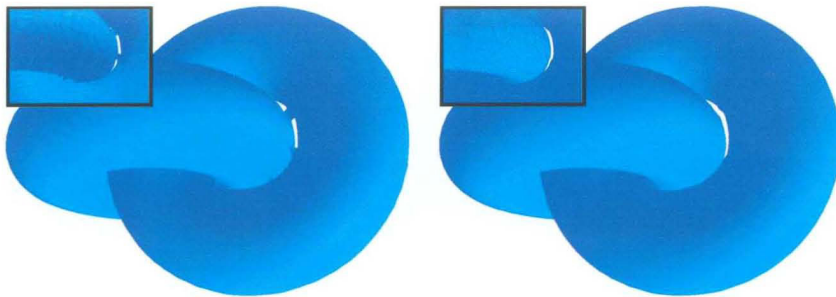


Figure 1. Implicit surface of linked tori generated by the classical Marching Cubes algorithm, and ours.

1.4. *Cube versus Tetrahedron*

Another range of techniques for isosurface generation is based on tetrahedra [Treece et al. 99], as opposed to cubes. Those methods guarantee the topological consistency, and have a small look-up table. However, they have many drawbacks. They generate many more triangles, with a weaker geometrical accuracy of the result: The tetrahedra's tilings are segmented even in obvious configuration, and the vertex position cannot be adjusted to fit the geometrical trilinear approximation as we do with cubes. Moreover, the ambiguity resolution that is hidden in those methods leads to slower algorithms, which are more difficult to speed up with hardware implementation. Our technique uses a complex look-up table that only needs to be stored, which enables our algorithm to be efficiently hardware accelerated. This technique allows a *topologically correct* result with a single entry cubical look-up table, by a low complexity algorithm. This is a significant practical improvement compared to the former state-of-the-art of isosurface tilers [Ning, Bloomenthal 93].

2. Marching Cubes with Topological Guarantees

2.1. *Marching Cubes*

The Marching Cubes method produces a triangle mesh of the preimage $f^{-1}(\alpha)$ of an isovalue α by a scalar function $f : \mathbb{R}^3 \rightarrow \mathbb{R}$. We will consider $\alpha = 0$ for the rest of the paper (considering $f - \alpha$). This scalar field is given by samples over a cuberille grid. The original method sweeps the grid, and tiles the surface cube per cube. Each vertex v of the cube is classified into positive and negative vertices, depending on whether $f(v)$ is greater than α or

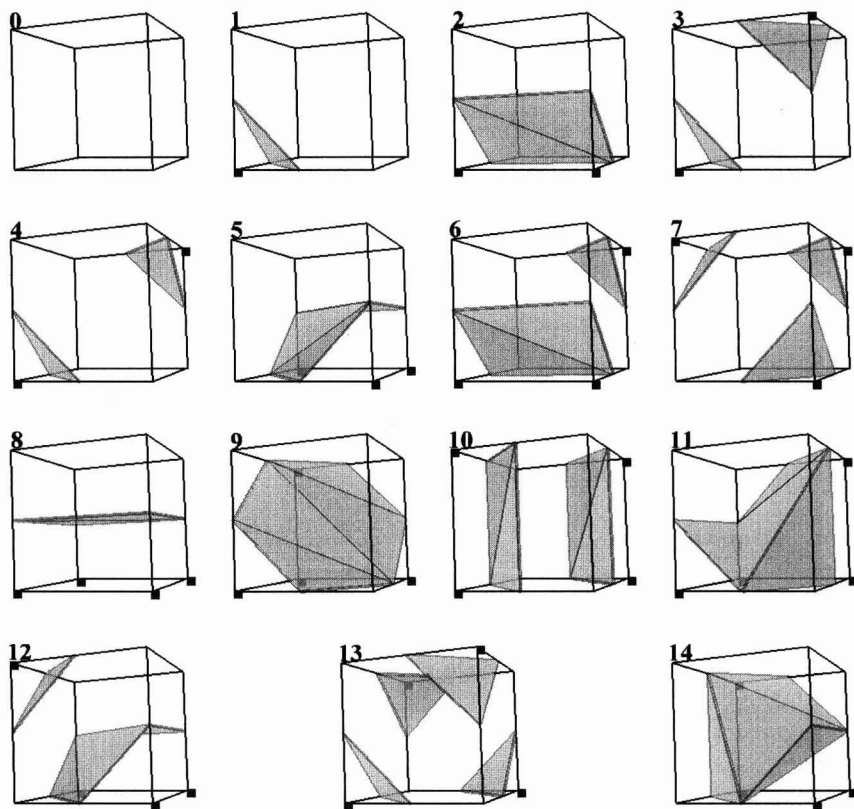


Figure 2. Original Marching Cubes' look-up table.

not. Thus, there are $2^8 = 256$ possible configurations of a cube. The usual implementation stores those 256 in a look-up table that encodes the tiling of the cube in each case (see Figure 2).

2.2. Correct Topology

However, this simple algorithm can lead to cracks, as shown in Figure 3. The same configuration can be tiled in various ways, and the 256-entry look-up table does not distinguish between those. Among the different tilings, some approximate a trilinear interpolation of the scalar field f over the cube. We will say a resulting mesh has the *correct topology* if it is homeomorphic to $F^{-1}(\alpha)$, where F is equal to f at the sample vertices, and trilinear over each

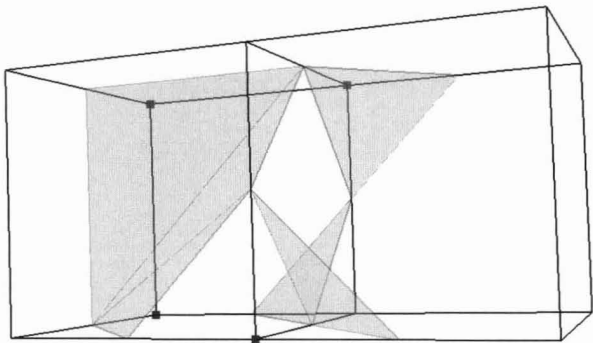


Figure 3. A crack occurring on an ambiguous face in between cases 12 and 3 with the 256-look-up table.

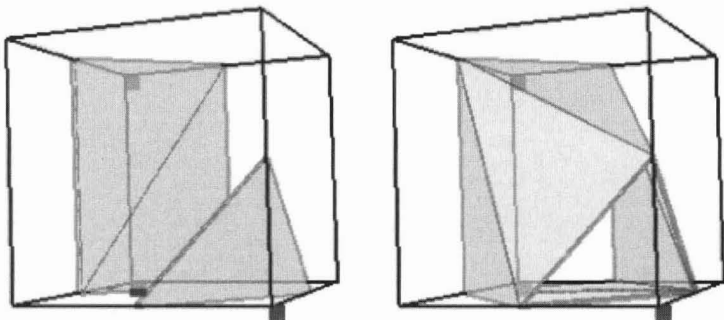


Figure 4. Two trilinear tilings of the sixth case, with the same resolution of faces' ambiguity.

cube of the grid. This allows avoiding cracks, by applying topological tests on ambiguous faces of a cube. The same test will be done on the adjacent cube, allowing a coherent transition from one cube to the other. Nielson and Hamann [Nielson, Hamann 91] introduced the usual face test to resolve those face ambiguities.

By resolving face ambiguities, we avoid cracks. Nevertheless, this does not guarantee the correct topology; as with the same cube configuration and the same resolution of ambiguous faces, there are topologically different trilinear interpolations (see Figure 4). Therefore, we also need to resolve internal ambiguity to guarantee the topology. The technique described in this paper guarantees the topology by providing an extended look-up table and an enhanced analysis of each cube.

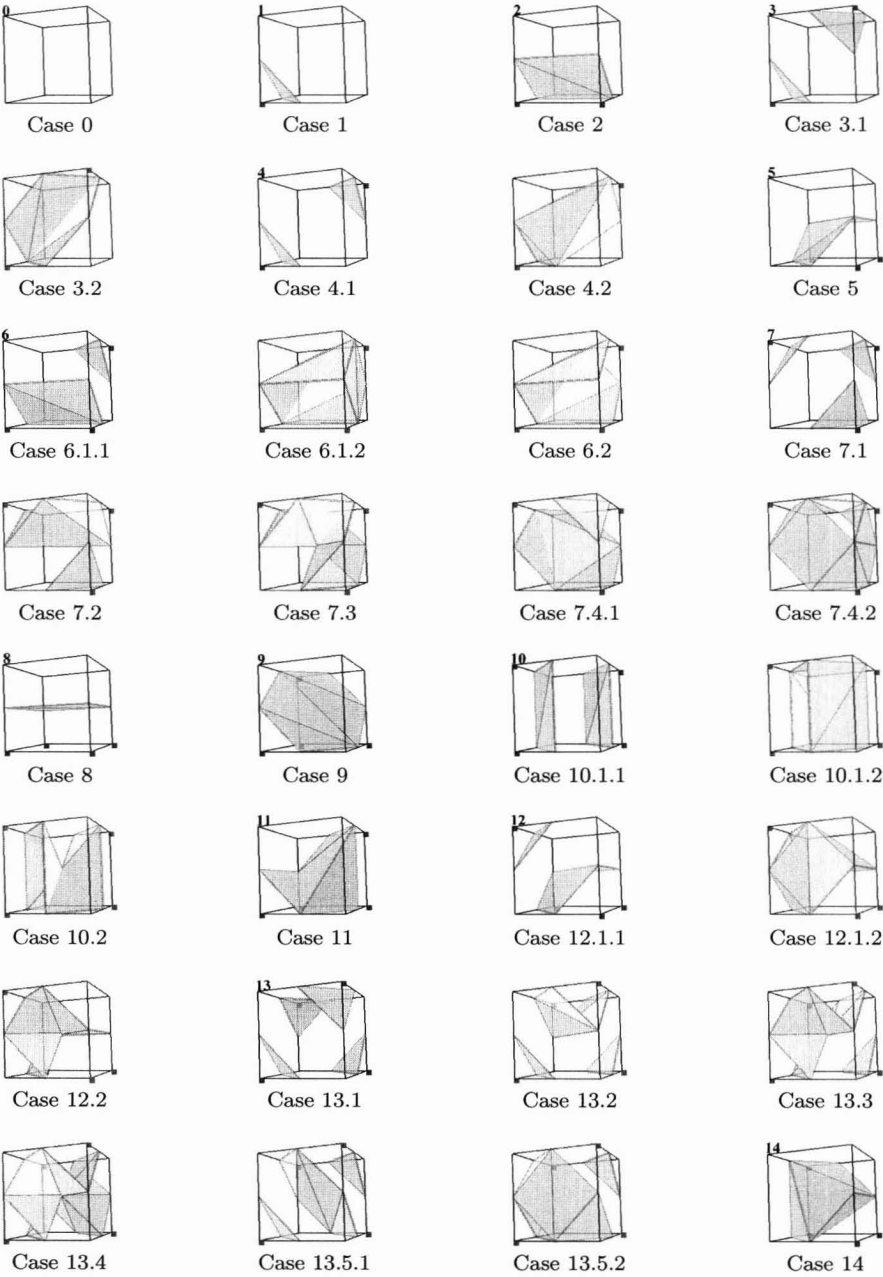


Figure 5. Chernyaev's look-up table.

2.3. *Marching Cubes 33*

Chernyaev described, with the Marching Cubes 33 [Chernyaev 95], the different possible topologies of a trilinear function over a cube. He gave a tiling for each case, adding some extra points for better geometrical approximation if necessary. He also proposed a method for resolving internal ambiguity, although it was not complete. We completed and enhanced this method, adding some tricks to avoid useless tests. We computed and tested the complete look-up table described by Chernyaev (see Figure 5).

3. Algorithm and Implementation

The algorithm goes through the following 4 steps:

1. Determine the case number and configuration (Section 3.1.).
2. Look up which faces are to be tested based on this configuration (Section 3.2.).
3. Determine the subcase based on the result of the face tests (Section 3.3.).
4. Look up the tiling of the cube for this subcase (Section 3.4.).

The simplicity of the algorithm relies on the look-up table, which is actually split into three tables:

- The *case table* maps each of the 256 possible configurations of a cube to one of the 15 cases of Figure 2, and to a specific number designating this configuration.
- The *test table* stores, for each configuration, the tests to be performed to resolve topological ambiguity.
- The *tiling table* stores the tiling for each configuration and subcase (there is no need for computing any geometrical transformation).

3.1. *Determining the Configuration*

The classical Marching Cubes look-up table has 256 entries (represented in Figure 2 by the 15 geometrically different cases¹). Each entry represents a different vertex configuration, given that a vertex is simply described by its sign (positive or negative).

¹i.e., that cannot be deduced by solid transformations.

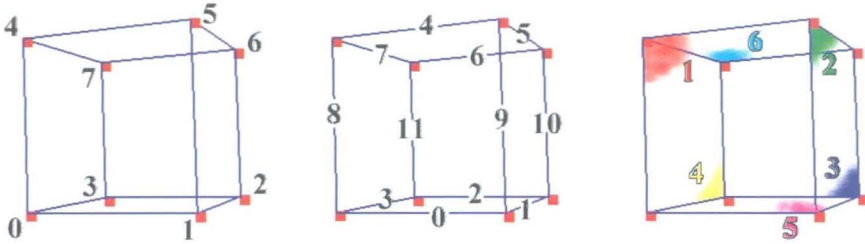


Figure 6. Labeling of vertices, edges, and faces: Vertex 0 has the lowest x , y , z coordinates, and vertex 6 the highest.

Each entry is identified by an 8-bit word b , as for the classical Marching Cubes: Its i^{th} bit is set to 1 (respectively, 0) if the i^{th} vertex of the cube is positive (respectively, negative). Figure 6 details the label of the vertices. The *case table* maps this 8-bit word b to the corresponding configuration of one of the 15 cases (see Figure 2). The configuration numbering is arbitrary.

For example, $b = 129$ means that only vertices 0 and 7 are positive, which is numbered in the case table as the third configuration of case 3 (see Figure 7).

3.2. Performing the Tests

To determine the topology of the graph of f inside each cube, it is not sufficient to know the sign of f over each vertex, even when f is trilinear. For example, Figure 7 shows two trilinear tilings of a cube with the same sign of the vertices. When two or more adjacent vertices of a face have the same sign, the topology of the isosurface on that face is obvious. Otherwise, this ambiguity can be resolved using the tests described in Section 4.1. When the positive vertices of a cube are connected by edges or through the faces, and also are the negative vertices, the topology of the interior of the cube is obvious. Otherwise, this ambiguity can be resolved using the method described in Section 4.2.

To know which tests are necessary to perform, a *test table* stores for each configuration of each ambiguous case (cases 3, 4, 6, 7, 10, 12, and 13) the label of the faces to be tested. The interior test sometimes requires an edge code further describing the configuration, which is stored at a specific position in the tiling table (see Section 4.2.). The labels of the edges and faces are detailed in Figure 6; face 7 stands for the interior.

For example, with $b = 129$, the third entry of the *test table* of case 3 is just '4', which means that face 4 is ambiguous (there is no adjacent vertex with the same sign) and has to be tested (see Figure 7).

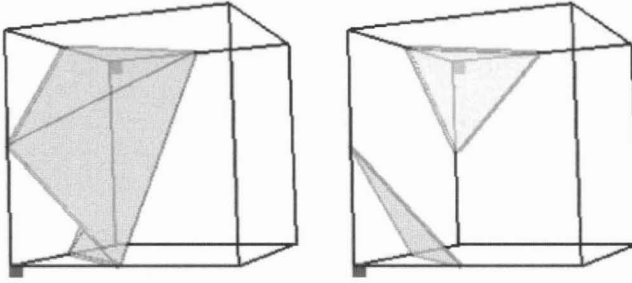


Figure 7. Case 3, configuration 3: two different tilings of the ambiguous face 4.

3.3. Determining the Subcase

The branching of those tests determines the subcase to be tiled (represented in Figure 5 by the 32 geometrically different cases). This branching is hard-coded for the simple cases (see Table 1). For complex cases (cases 7 and 13), other small tables are used to map the results of the tests to their corresponding subcase.

For example, with $b = 129$, the subcases of case 3 are determined by only one test, and is thus hard-coded. The cube will be tiled according to subcase 3.1 if the test is negative, and to subcase 3.2 otherwise (see Figures 5 and 7).

3.4. Tiling Each Cube

Each cube is tiled with triangles according to the *tiling table*. Each code of the tiling sequences identifies an edge of the cube (see Figure 6) on which a vertex of the cube's tiling will be computed. The twelfth code means interior vertex. The vertices of the final triangulation are computed by barycentric interpolations on the edges of the cube. Each group of three consecutive edge codes in the tiling table corresponds to a triangle.

For example, with $b = 129$, the third configuration of case 3 corresponds in the tiling table to

`{ 3,0,8,11,7,6,      7,8,6,8,0,6,3,11,6,3,6,0 }.`

Subcase 3.1 corresponds to the first sequence of the tiling table (2 triangles), and subcase 3.2 to the second one (4 triangles), as shown on Figure 7. As each tiling of the same subcase has the same number of triangles, those sequences are easily distinguished even if stored in the same table.

Case	Face tests			Interior Test	Subcase	# triangles
	1st	2nd	3rd			
0						0
1						1
2						2
3	-				3.1	2
	+				3.2	4
4	-				4.1	2
	+				4.2	6
5						3
6	-			-	6.1.1	3
	-			+	6.1.2	7
	+				6.2	5
7	-	-	-		7.1	3
	+	-	-		7.2	5
	-	+	-		7.2	5
	-	-	+		7.2	5
	+	+	-		7.3	9
	+	-	+		7.3	9
	-	+	+		7.3	9
	+	+	+	+	7.4.1	9
	+	+	+	-	7.4.2	9
8						2
9						4
10	+	+			10.1.1	4
	-	-		-	10.1.1	4
	-	-		+	10.1.2	8
	+	-			10.2	8
	-	+			10.2	8
11						4
12	+	+			12.1.1	4
	-	-		-	12.1.1	4
	-	-		+	12.1.2	8
	+	-			12.2	8
	-	+			12.2	8
13	45 subcases, testing all the 6 faces and eventually the interior					
14						4

Table 1. A reduced representation of the *test table*. Case 13 has 45 entries to map the results of all the possible tests to the right subcase.

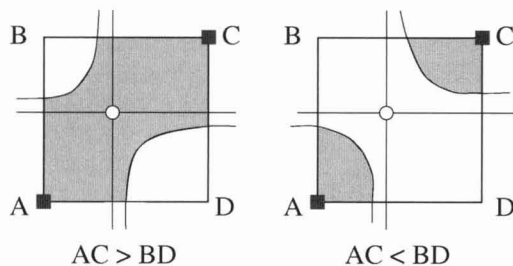


Figure 8. Face ambiguity test (Chernyaev's illustration [Chernyaev 95]).

4. Ambiguity Resolution

This section describes the methods used to test the faces and interior of a cube when the values at the vertices yield an ambiguous configuration. Those tests are used in Step 2 of the algorithm, to resolve those ambiguities on a face (see Section 4.1.) or on the interior of a cube (see Section 4.2.).

4.1. Resolution of Faces Ambiguities

Face ambiguity arises when two opposite vertices A and C of the face are positive, and the two others B and D are negatives (see Figures 7 and 8). We supposed the scalar field F is trilinear on each cube, thus, bilinear on each face. Therefore, $F^{-1}(\alpha)$ restricted to the face is a hyperbola (see Figure 8). Testing whether the center of the hyperbola is positive or negative (i.e., whether the positive vertices are connected inside the face) reduces (with $\alpha = 0$) to testing the sign of $F(A) \cdot F(C) - F(B) \cdot F(D)$.

The sign of the center of the hyperbola is the sign of the above expression if A is positive, and the opposite if A is negative. In addition, for some configuration, we want the opposite of the result. This is encoded by a negative face label in the test table. Therefore, the implemented test returns: $\text{sign}(\text{face_label} \cdot F(A) \cdot (F(A) \cdot F(C) - F(B) \cdot F(D)))$.

4.2. Resolution of Internal Ambiguities

An internal ambiguity arises when two diagonally opposite vertices A_0 and C_1 of a cube can be connected through the interior of the cube, creating a kind of tunnel (see Figures 4 and 9). Let's say those two vertices A_0 and C_1 are positive (the description holds for negative vertices also). We first resolve face ambiguity, according to Section 4.1. If there is a chain of positive vertices

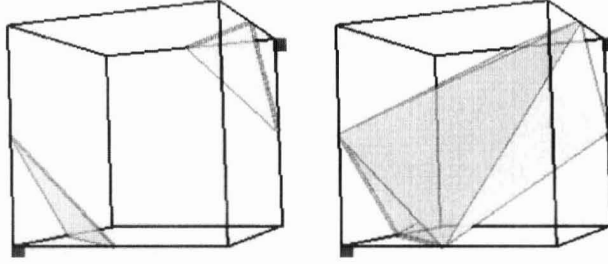


Figure 9. Case 4, configuration 0: two different tilings of the interior of a cube.

joining A_0 to C_1 , connected by edges or crossing ambiguous faces resolved as positive, then there is no internal ambiguity. Otherwise, we have to test if A_0 and C_1 are connected only through the cube.

Suppose A_0 and C_1 are connected through the interior of the cube. For $X = A, B, C, D$, let $X_t = t \cdot X_0 + (1 - t) \cdot X_1$. As F is trilinear, F cannot change sign more than once along the segment. Thus, there is a plane $P = (A_t, B_t, C_t, D_t)$, where the two edges A_0A_1 and C_1C_0 are connected inside the square A_t, B_t, C_t, D_t (see Figure 10). For each case, we must find the direction of P , compute the height t of P along the edge, and resolve the ambiguity inside the square A_t, B_t, C_t, D_t .

Cases 4 and 10 have enough symmetry to always choose the same direction, independently of the configuration. The height $t \in [0, 1]$ is the solution of Chernyaev's second order equation : $a \cdot t^2 + b \cdot t + c$, with :

$$\begin{aligned} a &= (A_1 - A_0) \cdot (C_1 - C_0) - (B_1 - B_0) \cdot (D_1 - D_0) \\ b &= C_0 \cdot (A_1 - A_0) + A_0 \cdot (C_1 - C_0) - D_0 \cdot (B_1 - B_0) - B_0 \cdot (D_1 - D_0) \\ c &= A_0 \cdot C_0 - B_0 \cdot D_0 \end{aligned}$$

For the other cases, the direction of P is encoded as an edge e inside one particular sequence of the tiling table: the seventeenth edge for cases 6, 7,

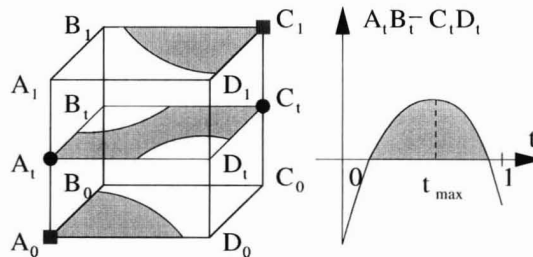


Figure 10. Internal ambiguity test for cases 4 and 10 (Chernyaev's illustration [Chernyaev 95]).

and 12; the second edge for the case 13.5. In that case, the height of the plane is the barycenter of the end vertices of e , weighted by F .

In both situations, the intersection of P with the cube is a square, with vertices A_t, B_t, C_t, D_t . There is no ambiguity inside this square if 0 or 1 vertex is positive (negative vertices are connected through the cube), if 3 or 4 vertices are positive (positive vertices are connected through the cube), or if 2 consecutive vertices are positive (no diagonal connection). In the other case, the square is ambiguous, and we resolve it in the same way as for face ambiguity.

5. Tips and Tricks

The vertices of the final mesh are interpolated along an edge. To avoid computing them more than once, they can all be computed first. To store them, we used three arrays, which assign respectively to each grid vertex an eventual index to the mesh vertex on the edge parallel to the x , y , and z axis.

The normal coordinates at each grid vertex $\vec{p}$ can be computed as

$$F(\vec{p} + \vec{\delta}) - F(\vec{p} - \vec{\delta}),$$

where $\vec{\delta}$ is the grid step along that coordinate. The normal at each point is then interpolated linearly.

A low-resolution extraction can be obtained by considering a lower resolution grid, i.e., taking into account every other vertex or every n -th vertex.

This algorithm is guaranteed to produce manifold meshes for any sample data, which allows working on previews with the same tools as on the final mesh.

This algorithm can be used for implicit surface tiling to construct fixed precision or exact result. In the latter case, it allows an economic use of exact arithmetic: When an evaluation of the surface inside a cube is ambiguous, the cube needs to be subdivided and the implicit function is evaluated again on the subdivision cubes. The tests we provided here can be substituted to avoid subdividing cubes, guaranteeing a manifold result. For example, an exact evaluation of the contour graph of F [Bajaj et al. 98] gives the number of connected components inside a cube. This topological information is a powerful test to distinguish between subcases.

References

- [Bajaj et al. 98] C. L. Bajaj, V. Pascucci, and D. Schikore. "Visualization of Scalar Topology for Structural Enhancement." In *IEEE Visualization '98*, edited by D. Ebert, H. Hagen, and H. Rushmeier, p. 51–58, Los Alamitos, CA: IEEE Press, 1998.

- [Bloomenthal 94] J. Bloomenthal. "An Implicit Surface Polygonizer." In *Graphics Gems IV*, p. 324–349. Boston: Academic Press, 1994.
- [Chernyaev 95] E. V. Chernyaev. "Marching cubes 33: Construction of topologically correct isosurfaces." Technical Report CERN CN 95-17, CERN, 1995. Available online at <http://wwwinfo.cern.ch/asdoc/psdir/mc.ps.gz>.
- [Davis et al. 02] J. Davis, S. Marschner, M. Garr, and M. Levoy. "Filling Holes in Complex Surfaces Using Volumetric Diffusion." In *First International Symposium on 3D Data Processing, Visualization, and Transmission*, 2002.
- [Gelder, Wilhelms 94] A. van Gelder and J. Wilhelms. "Topological Considerations in Isosurface Generation." *ACM Transactions on Graphics*, 13:4 (1994), 337–375.
- [Lorensen, Cline 87] W. E. Lorensen and H. Cline. "Marching Cubes: A High Resolution 3D Surface Construction Algorithm." In *Proc. SIGGRAPH '87, Computer Graphics* 21 (1987), 163–169.
- [Montani et al. 94] C. Montani, R. Scateni, and R. Scopigno. "A Modified Lookup Table for Implicit Disambiguation of Marching Cubes." *The Visual Computer*, 10:6 (1994), 353–355.
- [Natarajan 94] B. K. Natarajan. "On Generating Topologically Consistent Isosurfaces from Uniform Samples." *The Visual Computer: International Journal of Computer Graphics*, 11:1 (1994), 52–62.
- [Ning, Bloomenthal 93] P. Ning and J. Bloomenthal. "An Evaluation of Implicit Surface Tilers." *IEEE Computer Graphics and Applications*, 13:6 (1993), 33–41.
- [Nielson, Hamann 91] G. M. Nielson and B. Hamann. "The Asymptotic Decider: Resolving the Ambiguity in Marching Cubes." In *Proceedings of Visualization '91*, p. 29–38, 1991.
- [Tavares et al. 03] G. Tavares, R. Santos, H. Lopes, T. Lewiner, and A. W. Vieira. "Topological Reconstruction of Oil Reservoirs from Seismic Surfaces." In *International Association for Mathematical Geology*, 2003.
- [Treece et al. 99] G. Treece, R. Prager, and A. Gee. "Regularised Marching Tetrahedra: Improved Iso-Surface Extraction." *Computers and Graphics*, 23:4 (1999), 583–598.
- [Velho 96] L. Velho. "Simple and Efficient Polygonization of Implicit Surfaces." *journal of graphics tools*, 1:2 (1996), 5–25.
- [Weber et al. 02] G. H. Weber, G. Scheuermann, H. Hagen, and B. Hamann. "Exploring Scalar Fields Using Critical Isovalues." In *Proceedings of the IEEE Visualization 2002*, 2002. Hamann2002.pdf.

Web Information:

A C++ implementation together with the tables are available online at <http://www.acm.org/jgt/papers/LewinerEtAl03>. A small interface allows examining each entry of those tables.

Thomas Lewiner, Pontifical Catholic University of Rio de Janeiro, Departamento de Matematica, Rua Marquês de São Vicente, 225, Gávea 22453-900, Rio de Janeiro, Brazil (thomas.lewiner@polytechnique.org)

Hélio Lopes, Pontifical Catholic University of Rio de Janeiro, Departamento de Matematica, Rua Marquês de São Vicente, 225, Gávea 22453-900, Rio de Janeiro, Brazil (lopes@mat.puc-rio.br)

Antônio Wilson Vieira, Pontifical Catholic University of Rio de Janeiro, Departamento de Matematica, Rua Marquês de São Vicente, 225, Gávea 22453-900, Rio de Janeiro, Brazil (awilson@mat.puc-rio.br)

Geovan Tavares, Pontifical Catholic University of Rio de Janeiro, Departamento de Matematica, Rua Marquês de São Vicente, 225, Gávea 22453-900, Rio de Janeiro, Brazil (tavares@mat.puc-rio.br)

Received December 11, 2002; accepted September 16, 2003.